

# Laboratório de desenvolvimento de software

---

Revisão orientação a objetos

# Encapsulamento

---

- Em Java, o encapsulamento é implementado usando modificadores de acesso, que são palavras-chave que determinam o nível de acesso aos membros de uma classe (atributos e métodos).
- Os modificadores de acesso em Java são:
  - `public`: permite que qualquer classe possa acessar o membro.
  - `private`: permite que somente a própria classe possa acessar o membro.
  - `protected`: permite que as subclasses e classes do mesmo pacote possam acessar o membro.
  - `default (package)` : permite que as classes do mesmo pacote possam acessar o membro (é o modificador padrão, se nenhum outro modificador for especificado).

# Encapsulamento

---

- E para termos acesso a estes dados, precisamos também compreender a função dos *getters* e *setters*.
- Os *getters* e *setters* são métodos que permitem o acesso e a modificação dos atributos privados de uma classe, respeitando o conceito de encapsulamento.
- Eles são fundamentais na orientação a objetos, pois permitem que os dados sejam protegidos e acessados de forma controlada.
- Os *getters* são usados para obter o valor de um atributo, enquanto os *setters* são usados para alterar o valor de um atributo.

# Encapsulamento

---

- Em Java, os getters e setters seguem um padrão de nomenclatura, em que o nome do método começa com "get" ou "set", seguido do nome do atributo com a primeira letra em maiúscula.
- Por exemplo, se tivermos um atributo privado chamado "nome", o getter correspondente seria "getNome()" e o setter seria "setNome()".

# Encapsulamento - Exemplo

---

```
3 public class Pessoa {
4     private String nome;
5     private int idade;
6
7     public Pessoa(String nome, int idade) {
8         this.nome = nome;
9         this.idade = idade;
10    }
11
12    public String getNome() {
13        return nome;
14    }
15
16    public void setNome(String nome) {
17        this.nome = nome;
18    }
19
20    public int getIdade() {
21        return idade;
22    }
23
24    public void setIdade(int idade) {
25        this.idade = idade;
26    }
27 }
```

```
3 public class Pessoa {
4     private String nome;
5     private int idade;
6
7     public Pessoa(String nome, int idade) {
8         this.nome = nome;
9         this.idade = idade;
10    }
11
12    public String getNome() {
13        return nome;
14    }
15
16    public void setNome(String nome) {
17        this.nome = nome;
18    }
19
20    public int getIdade() {
21        return idade;
22    }
23
24    public void setIdade(int idade) {
25        this.idade = idade;
26    }
27 }
```

# Encapsulamento - Exemplo

---

- Neste exemplo, temos uma classe Pessoa com dois atributos privados (nome e idade) e os respectivos *getters* e *setters*.
- A ideia aqui é que os atributos não possam ser acessados ou modificados diretamente de fora da classe, apenas através dos métodos públicos.
- O construtor da classe é responsável por inicializar os atributos.
- Para acessar e modificar os atributos da classe, podemos usar os métodos *getters* e *setters*, como mostrado no próximo slide:

# Encapsulamento - Exemplo

---

```
3 public class Principal {
4
5     public static void main(String[] args) {
6         Pessoa pessoa = new Pessoa("Ricardo Frohlich", 22);
7         System.out.println(pessoa.getNome());
8         System.out.println(pessoa.getIdade());
9
10        pessoa.setNome("Rodrigo Ramos");
11        pessoa.setIdade(25);
12
13        System.out.println(pessoa.getNome());
14        System.out.println(pessoa.getIdade());
15    }
16
17 }
```



# Encapsulamento

---

- Os *getters* e *setters* são uma prática comum na programação orientada a objetos e são amplamente utilizados em diversas aplicações.
- Observe que, ao acessar os atributos nome e idade, estamos usando os métodos públicos `getNome()` e `getIdade()`, em vez de acessá-los diretamente.
- Isso garante que o encapsulamento seja respeitado e que os atributos não sejam acessados ou modificados diretamente.
- Eles permitem que os dados de uma classe sejam acessados e modificados de forma controlada e segura, contribuindo para a manutenção e evolução do código.

# Encapsulamento

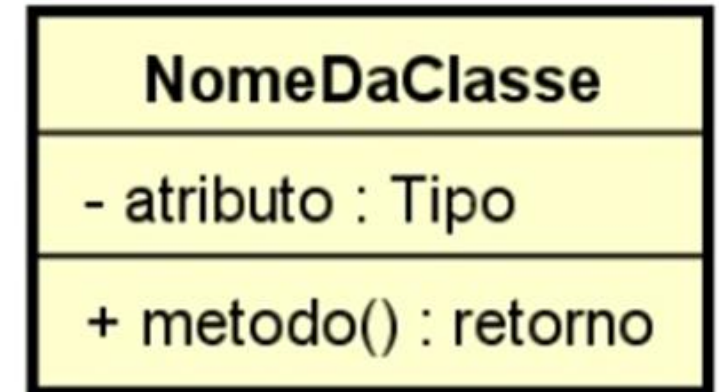
---

- O encapsulamento é um dos conceitos fundamentais da orientação a objetos e é essencial para garantir a integridade dos dados de uma classe.
- É importante lembrar que o encapsulamento não impede completamente o acesso aos atributos de uma, mas fornece uma camada de proteção adicional e ajuda a tornar o código mais robusto e seguro.

# Encapsulamento – Diagrama de classes

---

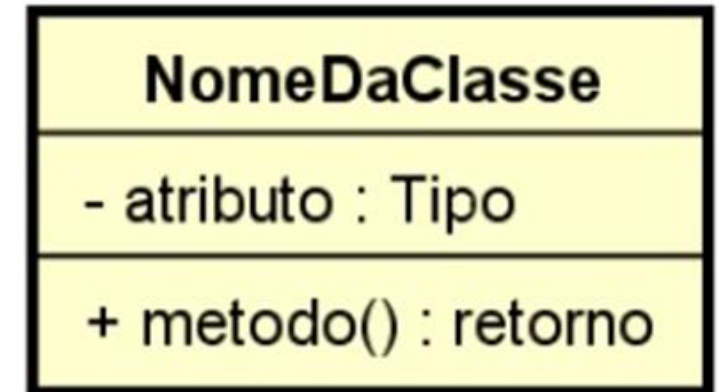
- E como compreender em um diagrama de classes?
- O Diagrama é dividido em três partes:
  - Nome da classe
  - Atributos
  - Métodos



# Encapsulamento – Diagrama de classes

---

- Atributos
  - <acesso> <nome> : <tipo>
- Métodos
  - <acesso> <nome> (<parâmetros>) : <tipo>
- <acesso> = público: sinal de “+”
- <acesso> = privado: sinal de “-”
- <acesso> = protegido: sinal de “#”
- <acesso> = default (package): sinal de “~”



# Jogo rápido

---

- Crie uma classe `Aluno` que possua os atributos `nome` e `nota1` e `nota 2`.
- Proteja os atributos utilizando encapsulamento.
- Crie os métodos *get* e *set* para cada atributo.
- Crie um método `calculaMedia` que calcule a média das notas dos alunos e retorne o resultado.

# Resolução

---

```
3 public class Aluno {
4     private String nome;
5     private float nota1, nota2;
6     public String getNome() {
7         return nome;
8     }
9     public void setNome(String nome) {
10         this.nome = nome;
11     }
12     public float getNota1() {
13         return nota1;
14     }
15     public void setNota1(float nota1) {
16         this.nota1 = nota1;
17     }
18     public float getNota2() {
19         return nota2;
20     }
21     public void setNota2(float nota2) {
22         this.nota2 = nota2;
23     }
24
25     public float calculaMedia() {
26         return (nota1+nota2)/2;
27     }
28 }
```

```
3 public class Aluno {
4     private String nome;
5     private float nota1, nota2;
6     public String getNome() {
7         return nome;
8     }
9     public void setNome(String nome) {
10         this.nome = nome;
11     }
12     public float getNota1() {
13         return nota1;
14     }
15     public void setNota1(float nota1) {
16         this.nota1 = nota1;
17     }
18     public float getNota2() {
19         return nota2;
20     }
21     public void setNota2(float nota2) {
22         this.nota2 = nota2;
23     }
24
25     public float calculaMedia() {
26         return (nota1+nota2)/2;
27     }
28 }
```

# Resolução

---

```
3 public class Principal2 {  
4     public static void main(String[] args) {  
5         Aluno a = new Aluno();  
6         a.setNome("Ricardo");  
7         a.setNota1(7);  
8         a.setNota2(6);  
9         System.out.println("A média final = "+a.calculaMedia());  
10    }  
11 }
```



# Herança

---

- Herança é uma forma de reutilização de software.
- Permite além de reutilização, manutenção simples e eficiente: alterações em cascata.
- Em Java a herança é observada através da palavra *extends*
- Java não permite herança múltipla: um artifício é utilizar uma classe, que herda de outra classe, que herda de outra classe e assim por diante.
- Exemplo:
  - classe 2 *extends* classe 1
  - classe 3 *extends* classe 2

# Herança

---

```
3 public class Carro {
4
5     protected String nome;
6
7     public String getNome() {
8         return nome;
9     }
10
11     public void setNome(String nome) {
12         this.nome = nome;
13     }
14
15     public void exhibeMsg() {
16         System.out.println("Estou na classe Carro\nO nome do carro é: "+nome);
17     }
18 }
```

# Herança

---

- A classe Onibus herda os atributos e os métodos da classe Carro

```
3 public class Onibus extends Carro{
4     protected String modelo;
5
6     public String getModelo() {
7         return modelo;
8     }
9
10    public void setModelo(String modelo) {
11        this.modelo = modelo;
12    }
13 }
```

# Herança

---

```
3 public class Principal {
4
5     public static void main(String[] args) {
6         Onibus o = new Onibus();
7         o.setNome("Marcopolo"); setNome -> Classe Carro
8         o.setModelo("OF-1519"); setModelo -> Classe Onibus
9
10        o.exibeMsg(); Este método está na classe Carro!
11
12        System.out.println("O nome do carro: "+o.getNome()); getNome -> Classe Carro
13        System.out.println("Modelo do onibus: "+o.getModelo()); getModelo -> Classe Onibus
14    }
15 }
```

# Herança

---

- Jogo rápido [1]:
  - Crie uma classe Animal com os seguintes atributos: nome, idade e som. Em seguida, crie uma classe Cachorro que herda de Animal e adiciona um atributo raça. Crie um método na classe Cachorro chamado latir() que exibe o som do cachorro.

# Herança

---

```
3 public class Animal {
4     private String nome;
5     private int idade;
6     protected String som;
7     public String getNome() {
8         return nome;
9     }
10    public void setNome(String nome) {
11        this.nome = nome;
12    }
13    public int getIdade() {
14        return idade;
15    }
16    public void setIdade(int idade) {
17        this.idade = idade;
18    }
19    public String getSom() {
20        return som;
21    }
22    public void setSom(String som) {
23        this.som = som;
24    }
25 }
```

# Herança

---

```
3 public class Cachorro extends Animal{
4     private String raca;
5
6     public String getRaca() {
7         return raca;
8     }
9
10    public void setRaca(String raca) {
11        this.raca = raca;
12    }
13
14    public void latir() {
15        System.out.println(som);
16    }
17 }
```

# Herança

---

```
3 public class Principal {  
4  
5     public static void main(String[] args) {  
6         Cachorro c = new Cachorro();  
7         /* atributos da classe Animal */  
8         c.setNome("Amarelo");  
9         c.setIdade(10);  
10        c.setSom("Au-Au");  
11  
12        /* atributos da classe Cachorro */  
13        c.setRaca("Vira-lata");  
14  
15        c.latir();  
16    }  
17 }
```



# Herança

---

- Uma classe Pessoa, clássica

```
3 public class Pessoa {  
4     protected String nome;  
5     protected int idade;  
6     public Pessoa(String nome, int idade) {  
7         this.nome = nome;  
8         this.idade = idade;  
9     }  
10  
11     public void exibeDados() {  
12         System.out.println("Nome: "+nome+"\nIdade: "+idade);  
13     }  
14 }
```

# Herança

---

- Uma classe Principal clássica

```
3 public class Principal {  
4  
5     public static void main(String[] args) {  
6         Pessoa p = new Pessoa("Ricardo", 40);  
7         p.exibeDados();  
8     }  
9 }
```

# Herança

---

- Jogo rápido [2]:
  - Quero criar uma classe PessoaJuridica que herde a classe Pessoa, como faço?
  - Atributos da classe PessoaJuridica:
    - String CNPJ
    - String socio
    - String dtAbertura

# Herança

---

```
3 public class PessoaJuridica extends Pessoa {
4     protected String CNPJ;
5     protected String socio;
6     protected String dtAbertura;
7     public PessoaJuridica(String nome, int idade, String CNPJ, String socio, String dtAbertura) {
8         super(nome, idade);
9         this.CNPJ = CNPJ;
10        this.socio = socio;
11        this.dtAbertura = dtAbertura;
12    }
13 }
```

- O comando `super` serve para chamar o construtor da superclasse, ou seja, da classe que é herdada.
- Ele sempre é chamado, mesmo quando não está explícito no código, quando for explicitado deve ser o primeiro item dentro do construtor.

# Herança

---

- No exemplo é chamado o super dentro do construtor onde é passado os parâmetros (nome e idade) da classe que é herdada (Pessoa).
- E abaixo são atribuído os atributos da classe PessoaJuridica.

# Polimorfismo

---

- A palavra polimorfismo vem do grego:
- poli = muitas, morphos = formas
- Ou seja: muitas formas
- Polimorfismo permite “programar no geral” ao invés de “programar no específico”.
- Exemplos de polimorfismo em Java são:
  - Sobrescrita de métodos
  - Sobrecarga de métodos

# Polimorfismo

---

- Sobrescrita de métodos (override): métodos com o mesmo nome, mesmos parâmetros e mesmo tipo de retorno, alterando apenas o comportamento do método.
  - Tudo no método é igual, só muda a implementação.
- Sobrecarga de métodos (overload): métodos com o mesmo nome, porém com parâmetros diferentes e/ou tipos de retorno diferentes.
- Dinâmico: em tempo de compilação é escolhido

# Polimorfismo - Sobrescrita

---

- Sobrescrita é o processo de fornecer uma implementação diferente para um método já definido em uma classe base.
- Isso é feito na classe derivada, que herda o método da classe base.
- A sobrescrita é realizada com a mesma assinatura de método da classe base, mas com uma implementação diferente.
- Isso permite que a classe derivada substitua o comportamento da classe base para o método em questão.



# Polimorfismo - Sobrescrita

---

```
3 public class Animal {  
4     public void fazerSom() {  
5         System.out.println("O animal está fazendo um som.");  
6     }  
7 }
```

```
3 public class Cachorro extends Animal {  
4     @Override  
5     public void fazerSom() {  
6         System.out.println("O cachorro está latindo.");  
7     }  
8 }
```

# Polimorfismo - Sobrescrita

---

```
3 public class SobrescritaExemplo {  
4     public static void main(String[] args) {  
5         Animal animal = new Animal();  
6         animal.fazerSom();  
7  
8         animal = new Cachorro();  
9         animal.fazerSom();  
10  
11        Cachorro cachorro = new Cachorro();  
12        cachorro.fazerSom();  
13  
14    }  
15 }
```

- Qual resultado acontece após cada chamada do método fazerSom()?

# Polimorfismo - Sobrescrita

---

- Jogo rápido:
  - Crie uma classe Pessoa com um método trabalhar(). Em seguida, crie uma classe Programador que herda da classe Pessoa e sobrescreve o método trabalhar() para imprimir "Programando...".

# Polimorfismo - Sobrescrita

---

- Jogo rápido:

```
3 public class Pessoa {  
4     public void trabalhar() {  
5         System.out.println("Eu só trabalho...");  
6     }  
7 }
```

```
3 public class Programador extends Pessoa {  
4     public void trabalhar() {  
5         System.out.println("Programando...");  
6     }  
7 }
```

● Captura Retangular

# Polimorfismo - Sobrescrita

---

- Jogo rápido:

```
5 public class Principal {  
6     public static void main(String[] args) {  
7         int op;  
8         Scanner sc = new Scanner(System.in);  
9         System.out.println("Digite 1 para criar uma pessoa e 2 para programador:");  
10        op = sc.nextInt();  
11        Pessoa p;  
12        if (op==1) {  
13            p = new Pessoa();  
14            p.trabalhar();  
15        }  
16        else if(op==2) {  
17            p = new Programador();  
18            p.trabalhar();  
19        }  
20    }  
21 }
```

# Polimorfismo - Sobrecarga

---

- A sobrecarga é o processo de fornecer várias implementações para um método com o mesmo nome, mas com diferentes assinaturas de parâmetros.
- Isso permite que a classe tenha vários métodos com o mesmo nome, mas que façam coisas diferentes, dependendo dos parâmetros que são passados.
- O Java decide qual método chamar com base nos parâmetros que são passados durante a chamada do método

# Polimorfismo - Sobrecarga

---

```
3 class Calculadora {  
4     public int somar(int x, int y) {  
5         return x + y;  
6     }  
7  
8     public int somar(int x, int y, int z) {  
9         return x + y + z;  
10    }  
11 }
```

# Polimorfismo - Sobrecarga

---

```
3 public class SobrecargaExemplo {  
4     public static void main(String[] args) {  
5         Calculadora calculadora = new Calculadora();  
6         int resultado1 = calculadora.somar(2, 3);  
7         int resultado2 = calculadora.somar(2, 3, 4);  
8         System.out.println("Resultado1: " + resultado1);  
9         System.out.println("Resultado2: " + resultado2);  
10    }  
11 }  
12
```



# Polimorfismo - Sobrecarga

---

- Jogo rápido: Faça a implementação para solucionar o erro do código gerado pela chamada abaixo:

```
3 public class SobrecargaExemplo {  
4     public static void main(String[] args) {  
5         Calculadora calculadora = new Calculadora();  
6         int resultado1 = calculadora.somar(2, 3);  
7         int resultado2 = calculadora.somar(2, 3, 4);  
8         System.out.println("Resultado1: " + resultado1);  
9         System.out.println("Resultado2: " + resultado2);  
10        → double resultado3 = calculadora.somar(3.5, 7);  
11        System.out.println("Resultado2: " + resultado3);  
12    }  
13 }
```

# Polimorfismo - Sobrecarga

---

- Jogo rápido: Faça a implementação para solucionar o erro do código gerado pela chamada abaixo:
- Resolução:

```
3  class Calculadora {  
4    public int somar(int x, int y) {  
5        return x + y;  
6    }  
7  
8    public int somar(int x, int y, int z) {  
9        return x + y + z;  
10   }  
11  
12  → public double somar (double x, double y) {  
13      return x + y;  
14  }  
15 }
```

# Polimorfismo - Sobrecarga

---

- E se a chamada for assim, em qual método da classe Calculadora ele executa?

```
3 public class SobrecargaExemplo {  
4     public static void main(String[] args) {  
5         Calculadora calculadora = new Calculadora();  
6         int resultado1 = calculadora.somar(2, 3);  
7         int resultado2 = calculadora.somar(2, 3, 4);  
8         System.out.println("Resultado1: " + resultado1);  
9         System.out.println("Resultado2: " + resultado2);  
10        → double resultado3 = calculadora.somar(3, 7);  
11        System.out.println("Resultado2: " + resultado3);  
12    }  
13 }
```

# Polimorfismo - Sobrecarga

---

- E se a chamada for assim, em qual método da classe Calculadora ele executa?
- Por mais que a chamada aguarde um retorno double, a chamada vai para o método que envia dois inteiros pois são os parâmetros enviados.

## Exercícios – Sobrescrita/Sobrecarga

---

- 1 - Crie uma classe FormaGeometrica com um método calcularArea(). Em seguida, crie uma classe Triangulo que herda da classe FormaGeometrica e sobrescreve o método calcularArea() para calcular a área do triângulo e imprimir o resultado.
- 2 - Crie uma classe Casa com um método calcularPreco(int tamanho) que retorna o preço da casa com base no tamanho em metros quadrados. Sobrecarregue o método calcularPreco() para aceitar um número de quartos e retornar o preço da casa com base no tamanho e no número de quartos.